

SQL Part 4: Subqueries, CASE, UNION, Text & Date Functions, NULL — Free Practice Guide

Beyond School · Ready to learn in a fun way! Anyone can learn — easy! Nothing to install, just a web browser.

Step 1: Open a free online SQL playground

- sqliteonline.com — choose "SQLite", paste, click Run.
- db-fiddle.com — pick "SQLite" as the database, paste and Run.
- sqlime.org — a simple SQLite editor in your browser.
- sqlbolt.com — free interactive lessons for extra practice.

Step 2: Paste this setup script and click Run

It creates the same library tables used in the video. If anything goes wrong, just run it again in a fresh playground.

```
CREATE TABLE books (
  id INTEGER PRIMARY KEY,
  title TEXT,
  author TEXT,
  year INTEGER,
  genre TEXT,
  pages INTEGER
);
INSERT INTO books VALUES
  (1, 'Harry Potter', 'J.K. Rowling', 1997, 'Fantasy', 309),
  (2, 'Matilda', 'Roald Dahl', 1988, 'Comedy', 240),
  (3, 'Wonder', 'R.J. Palacio', 2012, 'Drama', 310),
  (4, 'The Hobbit', 'J.R.R. Tolkien', 1937, 'Fantasy', 310),
  (5, 'Holes', 'Louis Sachar', 1998, 'Mystery', 233),
  (6, 'Charlotte''s Web', 'E.B. White', 1952, 'Classic', 192),
  (7, 'Dog Man', 'Dav Pilkey', 2016, 'Comedy', 240),
  (8, 'Coraline', 'Neil Gaiman', 2002, 'Fantasy', 162);

CREATE TABLE members (
  member_id INTEGER PRIMARY KEY,
  name TEXT,
  city TEXT,
  joined TEXT,
```

```
  email TEXT
);
INSERT INTO members VALUES
  (1, 'Aisha', 'Austin', '2023-01-15', 'aisha@mail.com'),
  (2, 'Ben', 'Denver', '2023-03-02', NULL),
  (3, 'Carlos', 'Austin', '2024-06-20', 'carlos@mail.com'),
  (4, 'Dana', 'Boston', '2024-09-10', NULL),
  (5, 'Eli', 'Denver', '2025-02-01', 'eli@mail.com');

CREATE TABLE loans (
  loan_id INTEGER PRIMARY KEY,
  book_id INTEGER,
  member_id INTEGER,
  loan_date TEXT,
  return_date TEXT
);
INSERT INTO loans VALUES
  (1, 1, 1, '2026-01-05', '2026-01-19'),
  (2, 3, 1, '2026-02-10', NULL),
  (3, 2, 2, '2026-01-20', '2026-02-03'),
  (4, 4, 3, '2026-03-01', '2026-03-15'),
  (5, 1, 3, '2026-04-02', NULL),
  (6, 7, 4, '2026-04-10', '2026-04-24'),
  (7, 7, 2, '2026-05-01', NULL),
  (8, 5, 4, '2026-05-15', NULL);
```

Step 3: Run every query from the video, in order

Paste one at a time, click Run, and compare with the expected result.

Subquery with a single value

```
SELECT title, pages FROM books WHERE pages > (SELECT AVG(pages)                FROM books);
```

```
Expected result:
title | pages
Harry Potter | 309
Wonder | 310
The Hobbit | 310
```

Subquery with IN

```
SELECT name FROM members WHERE member_id IN (   SELECT member_id FROM loans   WHERE return_date IS
NULL);
```

```
Expected result:
name
Aisha
Ben
Carlos
Dana
```

CASE: if this, then that

```
SELECT title, pages,   CASE WHEN pages >= 300 THEN 'Long'        WHEN pages >= 200 THEN 'Medium'
     ELSE 'Short'   END AS size FROM books;
```

```
Expected result:
title | pages | size
Harry Potter | 309 | Long
Matilda | 240 | Medium
Wonder | 310 | Long
The Hobbit | 310 | Long
Holes | 233 | Medium
Charlotte's Web | 192 | Short
Dog Man | 240 | Medium
```

```
Coraline | 162 | Short
```

CASE + GROUP BY

```
SELECT CASE WHEN pages >= 300 THEN 'Long'             WHEN pages >= 200 THEN 'Medium'
ELSE 'Short' END AS size,        COUNT(*) AS books FROM books GROUP BY size;
```

```
Expected result:
size | books
Long | 3
Medium | 3
Short | 2
```

UNION

```
SELECT genre FROM books WHERE year < 1990 UNION SELECT genre FROM books WHERE year > 2010;
```

```
Expected result:
genre
Classic
Comedy
Drama
Fantasy
```

UNION ALL

```
SELECT genre FROM books WHERE year < 1990 UNION ALL SELECT genre FROM books WHERE year > 2010;
```

```
Expected result:
genre
Comedy
Fantasy
Classic
Drama
Comedy
```

UPPER and LENGTH

```
SELECT UPPER(title) AS shout,        LENGTH(title) AS letters FROM books ORDER BY letters DESC;
```

```
Expected result:
shout | letters
CHARLOTTE'S WEB | 15
HARRY POTTER | 12
THE HOBBIT | 10
CORALINE | 8
MATILDA | 7
DOG MAN | 7
WONDER | 6
HOLES | 5
```

|| and SUBSTR

```
SELECT title || ' by ' || author AS label,        SUBSTR(author, 1, 1) AS initial FROM books LIMIT
5;
```

```
Expected result:
label | initial
Harry Potter by J.K. Rowling | J
Matilda by Roald Dahl | R
Wonder by R.J. Palacio | R
The Hobbit by J.R.R. Tolkien | J
Holes by Louis Sachar | L
```

Date functions

```
SELECT loan_id, loan_date,         strftime('%m', loan_date) AS month,         date(loan_date, '+14
days') AS due FROM loans;
```

```
Expected result:
loan_id | loan_date | month | due
1 | 2026-01-05 | 01 | 2026-01-19
2 | 2026-02-10 | 02 | 2026-02-24
3 | 2026-01-20 | 01 | 2026-02-03
4 | 2026-03-01 | 03 | 2026-03-15
```

```
5 | 2026-04-02 | 04 | 2026-04-16
6 | 2026-04-10 | 04 | 2026-04-24
7 | 2026-05-01 | 05 | 2026-05-15
8 | 2026-05-15 | 05 | 2026-05-29
```

Days between dates

```
SELECT loan_id,         julianday(return_date)       - julianday(loan_date) AS days_kept FROM loans
WHERE return_date IS NOT NULL;
```

```
Expected result:
loan_id | days_kept
1 | 14.0
3 | 14.0
4 | 14.0
6 | 14.0
```

IS NULL

```
SELECT name, email FROM members WHERE email IS NULL;
```

```
Expected result:
name | email
Ben | NULL
Dana | NULL
```

COALESCE

```
SELECT name,         COALESCE(email, 'no email yet') AS contact FROM members;
```

```
Expected result:
name | contact
Aisha | aisha@mail.com
Ben | no email yet
Carlos | carlos@mail.com
Dana | no email yet
Eli | eli@mail.com
```

NOT IN subquery

```
SELECT title FROM books WHERE id NOT IN (    SELECT book_id FROM loans);
```

```
Expected result:
title
Charlotte's Web
Coraline
```

CASE with NULL

```
SELECT loan_id,    CASE WHEN return_date IS NULL         THEN 'Still out'         ELSE 'Returned'
END AS status FROM loans;
```

```
Expected result:
loan_id | status
1 | Returned
2 | Still out
3 | Returned
4 | Returned
5 | Still out
6 | Returned
7 | Still out
8 | Still out
```

LOWER and REPLACE

```
SELECT title,    LOWER(REPLACE(title, ' ', '-')) AS web_name FROM books;
```

```
Expected result:
title | web_name
Harry Potter | harry-potter
Matilda | matilda
Wonder | wonder
The Hobbit | the-hobbit
Holes | holes
Charlotte's Web | charlotte's-web
```

```
Dog Man | dog-man
Coraline | coraline
```

COUNT skips NULL

```
SELECT COUNT(*) AS members,        COUNT(email) AS with_email FROM members;
```

```
Expected result:
members | with_email
5 | 3
```

Step 4: Challenges (try first!)

C1. Use CASE to label each loan as Returned or Still out.

C2. Which books have fewer pages than the longest book? (subquery)

C3. Show every member name in capitals with their city glued on.

C4. Show each loan's return date, or 'not yet' if missing.

Answers

C1. Use CASE to label each loan as Returned or Still out.

```
SELECT loan_id,
  CASE WHEN return_date IS NULL THEN 'Still out' ELSE 'Returned' END AS status
FROM loans;
```

```
Expected result:
loan_id | status
1 | Returned
2 | Still out
3 | Returned
4 | Returned
5 | Still out
6 | Returned
7 | Still out
8 | Still out
```

C2. Which books have fewer pages than the longest book? (subquery)

```
SELECT title, pages FROM books WHERE pages < (SELECT MAX(pages) FROM books);
```

```
Expected result:
title | pages
Harry Potter | 309
Matilda | 240
Holes | 233
Charlotte's Web | 192
Dog Man | 240
Coraline | 162
```

C3. Show every member name in capitals with their city glued on.

```
SELECT UPPER(name) || ' - ' || city AS label FROM members;
```

```
Expected result:
label
AISHA - Austin
BEN - Denver
CARLOS - Austin
DANA - Boston
ELI - Denver
```

C4. Show each loan's return date, or 'not yet' if missing.

```
SELECT loan_id, COALESCE(return_date, 'not yet') AS returned FROM loans;
```

```
Expected result:
loan_id | returned
1 | 2026-01-19
2 | not yet
3 | 2026-02-03
```

```
4 | 2026-03-15
5 | not yet
6 | 2026-04-24
7 | not yet
8 | not yet
```

Keep going!

Watch the whole free SQL course on Beyond School, starting with SQL Part 1: https://youtu.be/72JvYu2OtiA
Beyond School — Ready to learn in a fun way!